

Experiment No.	7
Experiment Title	Dimensionality Reduction and Model Evaluation With and Without PCA
Student Name	Hemnath D
Register Number	3122247001022
Faculty	Dr. Poreddy Ajay Kumar Reddy
Submission Date	30 August 2026

1 Objective

To study how Principal Component Analysis (PCA) affects the performance of ten classifiers on a high-dimensional image dataset, and, as an extension, to check whether the same pattern holds for regression models on a continuous target. For both tasks, models are tuned with a grid search and evaluated with 5-fold cross-validation, once on the original feature space and once on a PCA-reduced feature space, so that the two settings can be compared fold-for-fold.

2 Problem Statement

Given a set of labelled feature vectors, the goal is to compare classifier behaviour with and without dimensionality reduction. The classification task uses the **Digits** dataset: each sample is a 64-dimensional vector of pixel intensities from an 8×8 grayscale scan of a handwritten digit, and the output is the digit class (0–9). As an extension, the same question is asked for a regression task using the **Diabetes** dataset: each sample is a 10-dimensional vector of baseline physiological measurements, and the output is a continuous disease-progression score. In both cases, the objective is to quantify whether reducing the input dimensionality with PCA helps, hurts or makes no meaningful difference to model accuracy and stability, and whether that answer depends on the type of model being used.

3 Dataset Description

3.1 Classification Dataset: Digits

Dataset Name	Digits (handwritten digit recognition)
Dataset Source	UCI ML Repository; bundled with scikit-learn as <code>load_digits()</code>
Number of Samples	1797
Number of Features	64 numerical, integer-valued pixel intensities (0–16), from an 8×8 grayscale image
Number of Classes	10 (digits 0–9), reasonably balanced (174–183 samples per class)
Missing Values	None (0 missing entries, 0 duplicate rows)
Train-Test Split	80% train (1437 samples) / 20% test (360 samples), stratified on digit label

This dataset was chosen for the primary PCA study because it is genuinely high-dimensional (64 features) and its neighbouring pixels are strongly correlated with one another, which is

exactly the setting in which PCA is expected to be most useful and where its effect is easiest to see and visualise.

3.2 Regression Dataset (Extension): Diabetes

Dataset Name	Diabetes (disease progression)
Dataset Source	UCI-derived; bundled with scikit-learn as <code>load_diabetes()</code>
Number of Samples	442
Number of Features	10 numerical features: age, sex, BMI, average blood pressure and six blood-serum measurements (<code>s1–s6</code>)
Target	Continuous disease-progression score one year after baseline, range $[25, 346]$
Missing Values	None (0 missing entries, 0 duplicate rows)
Train-Test Split	80% train (353 samples) / 20% test (89 samples)

The prescribed assignment is framed entirely around classifiers; this regression dataset was added to check whether PCA’s effect on model performance is specific to the high-dimensional, strongly-correlated classification setting, or whether it also shows up on a small, already compact regression feature set where several blood-panel measurements are naturally correlated.

4 Exploratory Data Analysis

4.1 Part A: Digits (Classification)

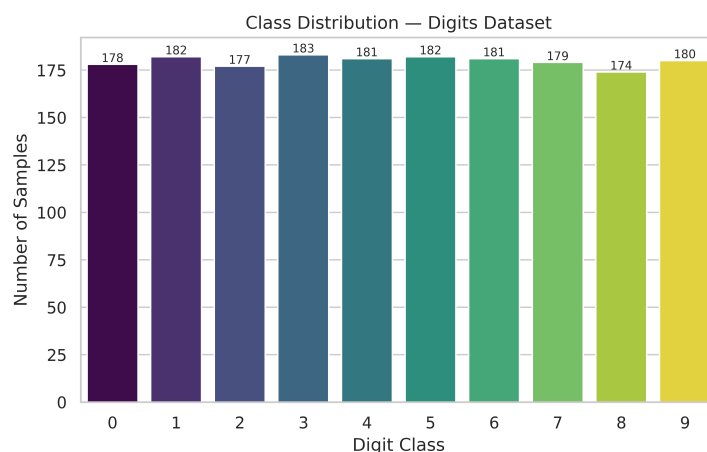


Figure 1: Class distribution across the ten digit classes

Inference: The dataset is close to perfectly balanced, with each digit class holding between roughly 174 and 183 samples. Because no class dominates, accuracy is a fair headline metric here, though macro-F1 is still tracked throughout to catch any per-class weaknesses that a simple average could hide. The near-uniform distribution also means stratified splitting is more of a safety net than a necessity, but it was used regardless to keep the fold-wise comparisons as clean as possible.

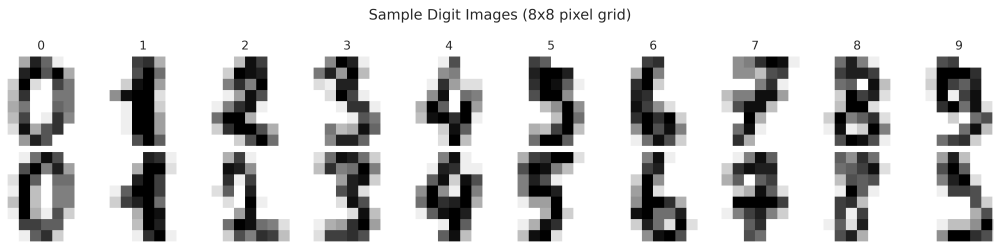


Figure 2: Two sample images per digit class

Inference: The raw 8×8 scans confirm that individual digits are visually distinguishable even at this very low resolution, though some pairs (for instance 4 and 9, or 3 and 8) share enough stroke structure that a model relying on only a handful of pixels could plausibly confuse them. This motivates using the full 64-pixel feature set, or a PCA projection that still retains the bulk of that information, rather than a small hand-picked feature subset.

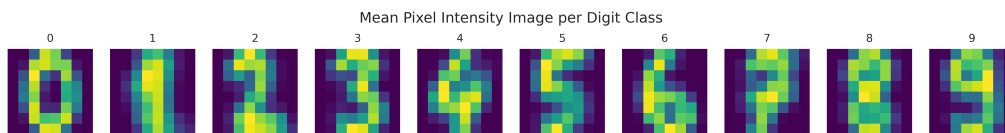


Figure 3: Mean pixel-intensity image per digit class

Inference: Averaging over all samples of a given class produces a clean, recognisable digit shape for every class, which indicates that the handwriting styles in the dataset are consistent enough that class-level structure survives averaging rather than being washed out by noise. This is a good sign for any linear or distance-based classifier: if the class means are already visually separable, then a model that captures roughly linear or Euclidean relationships between pixels has a real chance of separating classes well.

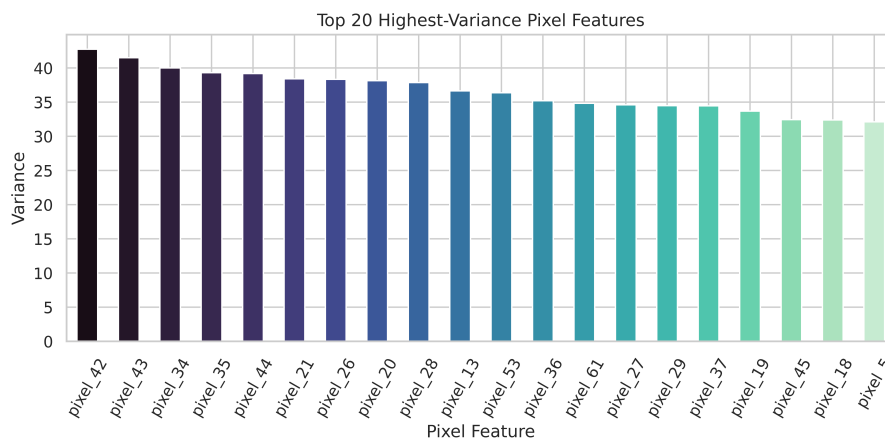


Figure 4: Top 20 highest-variance pixel features

Inference: Variance is concentrated in the central pixels of the 8×8 grid, while the border pixels (corners in particular) carry almost no variance since they are nearly always zero (unwritten background). This uneven variance distribution is precisely what PCA exploits: a handful of components aligned with the high-variance central region can summarise most of the useful signal, while the near-constant border pixels contribute very little to the first several principal components.

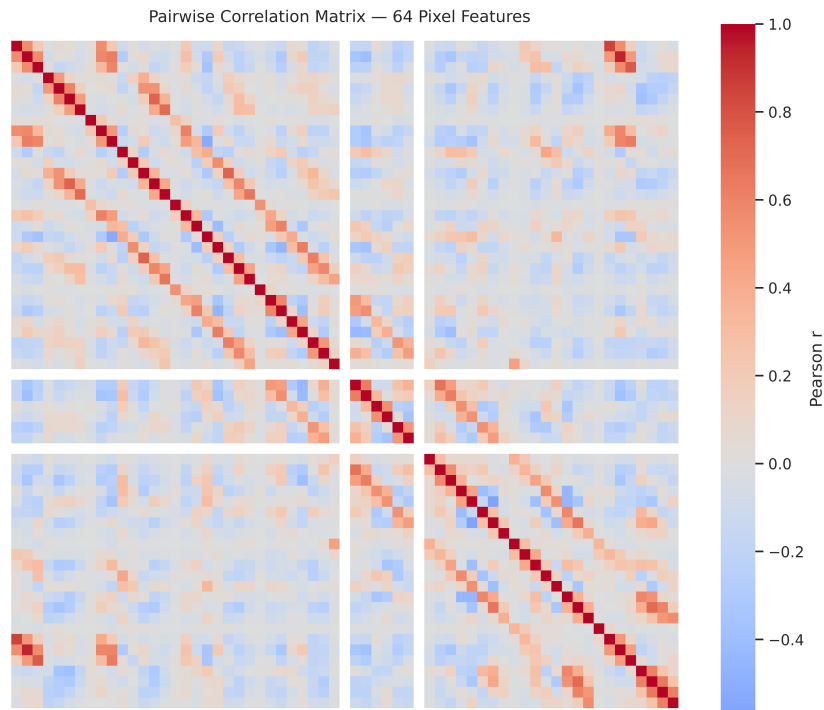


Figure 5: Pairwise correlation matrix across all 64 pixel features

Inference: Clear blocks of strong positive correlation appear between spatially adjacent pixels, which makes sense since a pen stroke covers several neighbouring pixels at once rather than a single isolated one. This redundancy is the second condition under which PCA is expected to help: not only is variance concentrated in a subset of pixels, those pixels are also highly correlated with each other, so a rotation of the feature space (PCA) can express nearly the same information in far fewer, decorrelated dimensions.

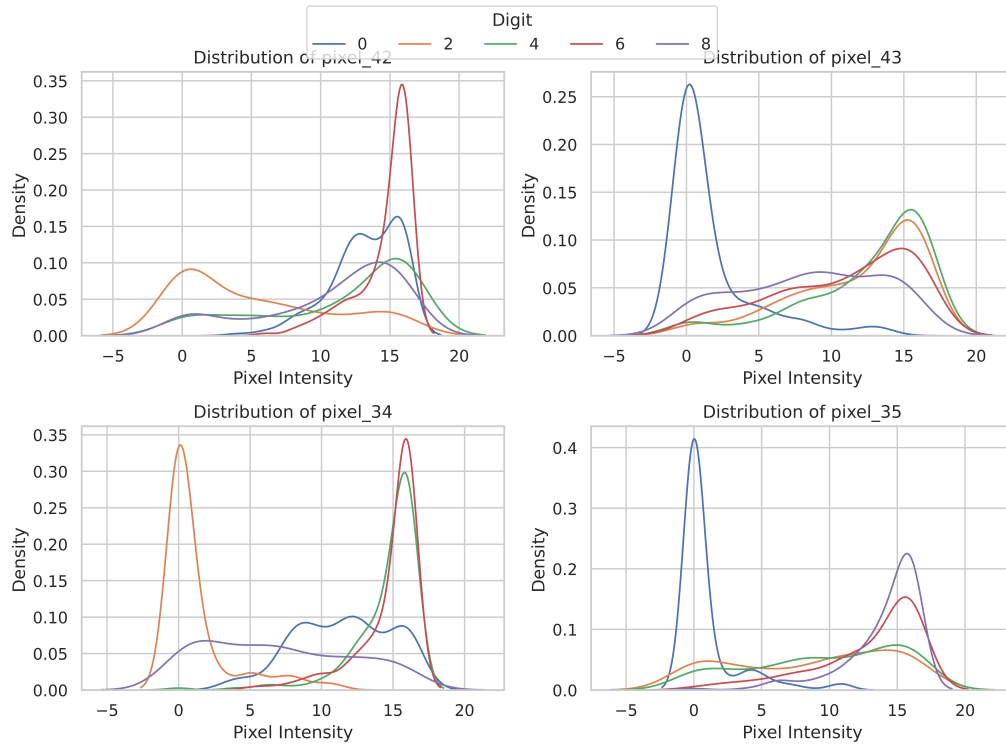


Figure 6: Kernel density estimates of the four highest-variance pixel features, split by digit class (even digits shown)

Inference: The four highest-variance pixels show visibly different intensity distributions across digit classes, with some classes concentrated near zero (pixel unused for that digit) and others spread across mid-to-high intensities. None of the four features cleanly separates all ten classes on its own, which is expected and is exactly why a multivariate model (and, for the With-PCA setting, a multivariate projection) is needed rather than a simple threshold rule on a single pixel.

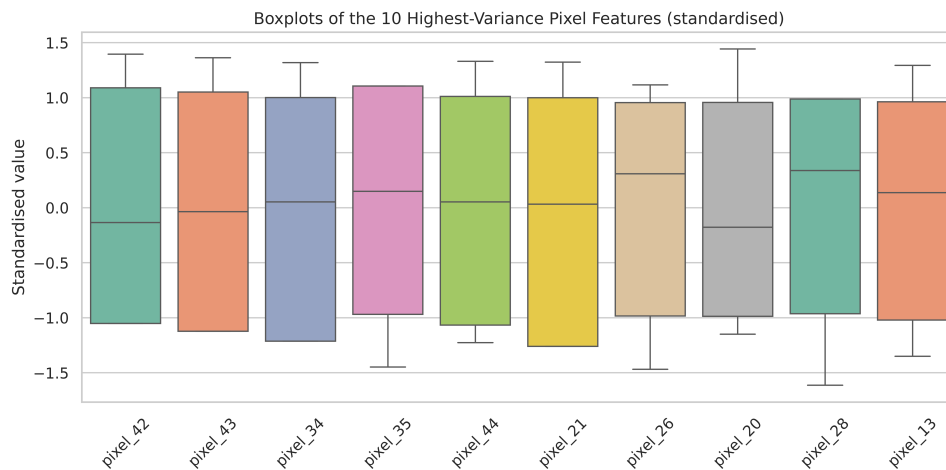


Figure 7: Boxplots of the ten highest-variance pixel features (standardised)

Inference: A modest number of points fall outside the whiskers for most of the plotted features, which is expected given natural handwriting variability (a stroke that is unusually thick, thin or off-centre for a given digit). These points were kept rather than removed, since they represent legitimate handwriting variation rather than data-entry errors, and since several

of the models compared here (tree ensembles and PCA itself when computed on standardised data) are not strongly disrupted by a small fraction of moderate outliers.

4.2 Part B: Diabetes (Regression Extension)

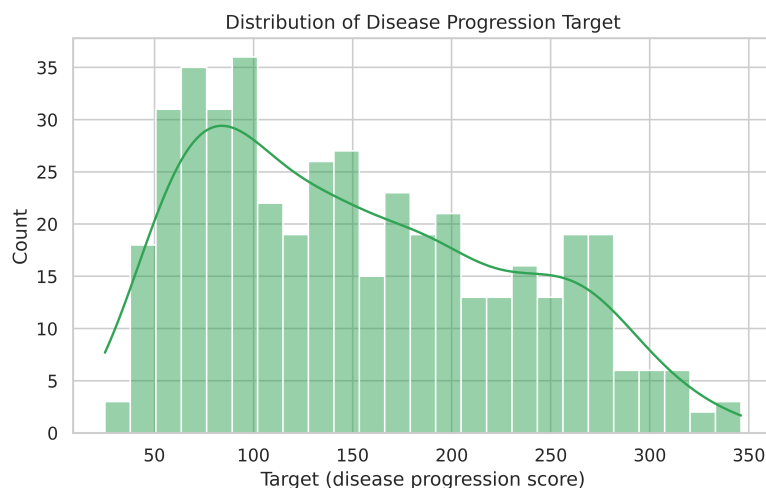


Figure 8: Distribution of the disease-progression target

Inference: The target is right-skewed with the bulk of patients clustered in the lower-to-middle range and a longer tail toward higher progression scores. This mild skew is not severe enough to require a target transformation, but it does mean that a small number of higher-progression patients will disproportionately influence squared-error-based metrics like RMSE, which is worth keeping in mind when comparing models later.

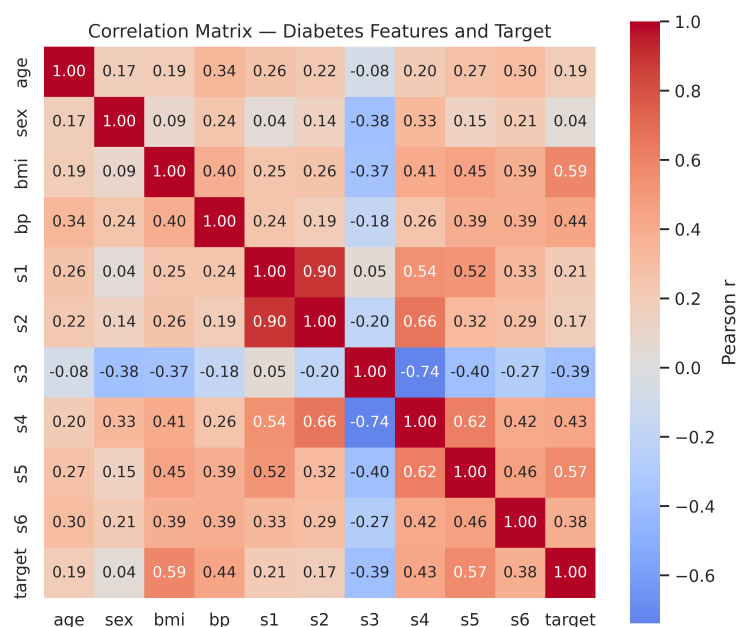


Figure 9: Correlation matrix of the ten Diabetes features and the target

Inference: BMI and the s5 blood-serum measurement show the strongest positive correlation with the target, while several of the blood-serum features (s1, s2, s3, s4) are strongly

correlated with each other, as would be expected from a lipid panel measuring related quantities. This internal redundancy among the serum features is the main justification for including PCA in this extension, even though the feature set is already fairly compact.

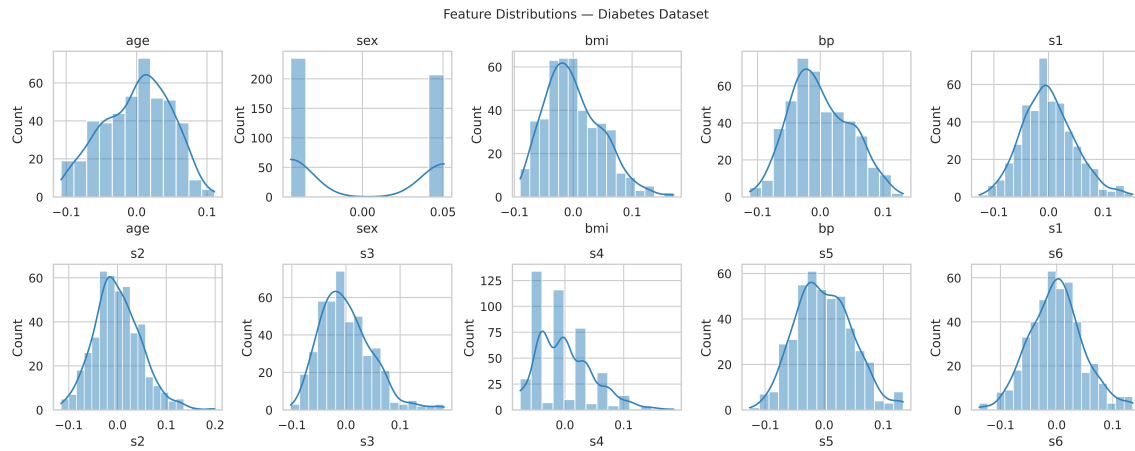


Figure 10: Distributions of all ten Diabetes features

Inference: Most features are roughly bell-shaped after the built-in mean-centring applied by scikit-learn’s loader, with `sex` appearing as a two-value categorical-like feature rather than a continuous one. None of the features show extreme skew or multi-modality, so no additional transformation was applied beyond the standard scaling used for every model.

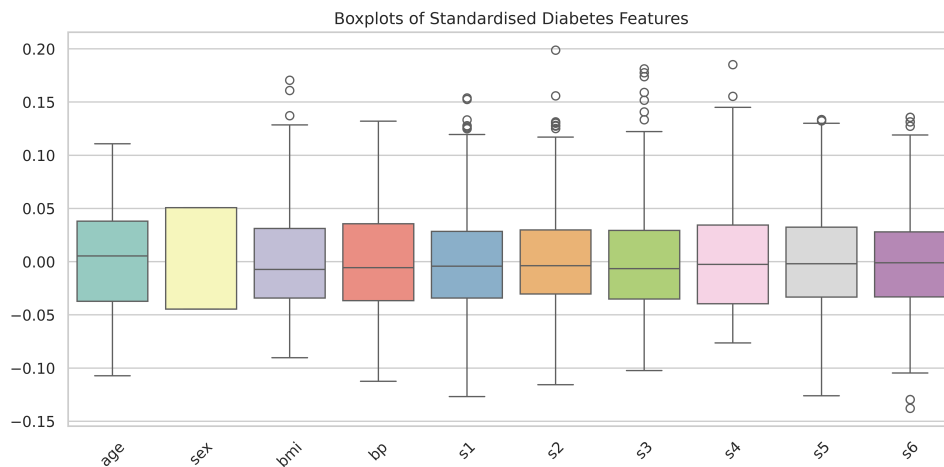


Figure 11: Boxplots of the standardised Diabetes features

Inference: A small number of outlying points appear for several serum measurements, but none of the features show a heavy concentration of extreme values. As with Part A, these points were retained rather than removed, since the dataset is a well-established benchmark and the outliers likely correspond to genuine physiological variation between patients rather than recording errors.

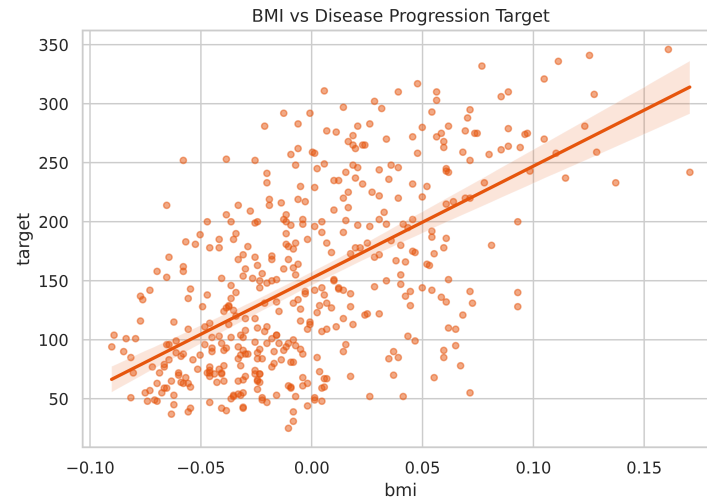


Figure 12: BMI versus disease-progression target, with a linear fit

Inference: BMI shows a clear positive linear trend with the target, consistent with its high correlation coefficient in the correlation matrix. The relationship is noisy rather than deterministic, since BMI alone leaves a lot of variance in the target unexplained, which anticipates the moderate ($R^2 \approx 0.4\text{--}0.5$) performance ceiling seen for every model in the results section, since no single feature or their linear combination captures the full picture.

5 Data Preprocessing

The following steps were applied identically to both datasets, before any model was trained:

- **Missing value handling:** Neither dataset contained missing entries or duplicate rows, so no imputation or deduplication was necessary.
- **Encoding:** All features in both datasets are already numeric, so no categorical encoding was required; the Digits target is an integer class label and the Diabetes target is a continuous score, so neither needed label encoding.
- **Feature scaling:** All features were standardised with `StandardScaler` (zero mean, unit variance), fit only on the training split and then applied to the test split, to avoid data leakage and to put every feature on a comparable scale before PCA and before distance-based models such as KNN and SVM.
- **Train-test split:** An 80/20 split was used for both datasets (stratified on the class label for Digits; a plain random split for the continuous Diabetes target).
- **Feature engineering:** No new features were engineered for either dataset; PCA itself is the only dimensionality transformation applied, and it is treated as a separate “With-PCA” branch of the pipeline rather than a permanent preprocessing step.

Listing 1: Standardisation and stratified train-test split (Digits)

```

1 X_train, X_test, y_train, y_test = train_test_split(
2     X, y, test_size=0.20, random_state=42, stratify=y
3 )
4
5 scaler = StandardScaler()
6 X_train_s = scaler.fit_transform(X_train) # fit on train only
7 X_test_s = scaler.transform(X_test) # avoid leakage

```

6 Principal Component Analysis

For both datasets, PCA was fit on the standardised training features, and the number of components was chosen as the smallest number that retains at least 95% of the cumulative explained variance, a conventional threshold that keeps almost all of the discriminative signal while discarding the low-variance, noise-dominated components.

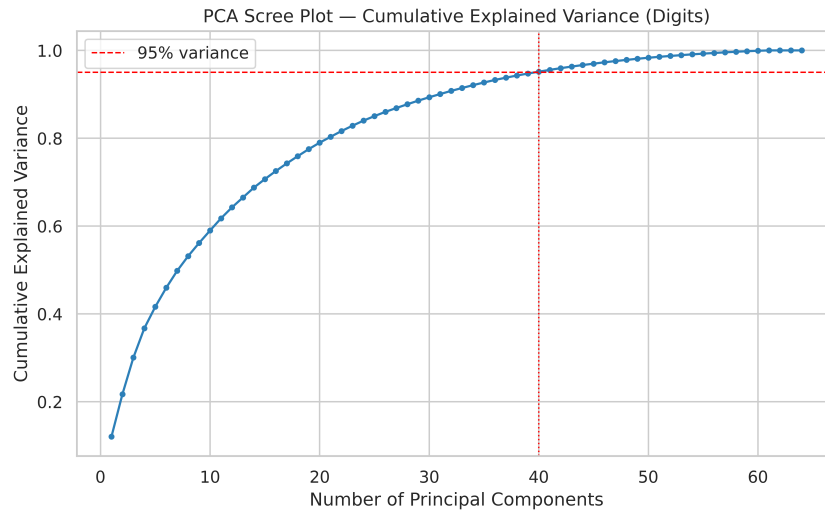


Figure 13: PCA scree plot: cumulative explained variance (Digits)

Inference: 90% of the variance is captured in the first 31 components and 95% in the first 40, out of the original 64, a meaningful but not dramatic reduction, since even the lower-variance pixel components still carry some class-discriminative signal. The curve’s early steep rise followed by a long, shallow tail is typical of image-like data, where a moderate number of components captures most of the shape information while the remaining components mostly encode fine-grained noise and idiosyncratic stroke details.

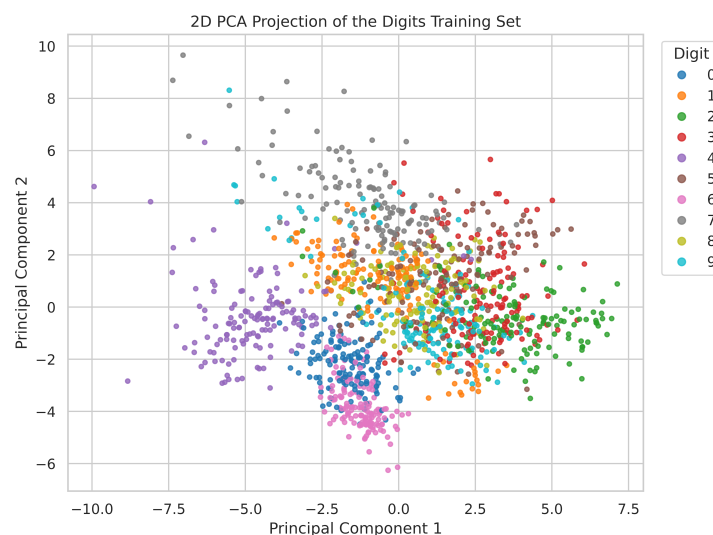


Figure 14: 2D PCA projection of the Digits training set, coloured by class

Inference: Even with only the first two components, several digit classes (notably 0, 4, and 6) form visibly separate clusters, while others (3, 5, 8, 9) overlap more heavily. This confirms that a large share of the class-relevant structure survives dimensionality reduction,

and it foreshadows the classifier results: models that rely on this kind of global geometric separation (KNN, SVM, Logistic Regression) should do reasonably well even in the reduced space.

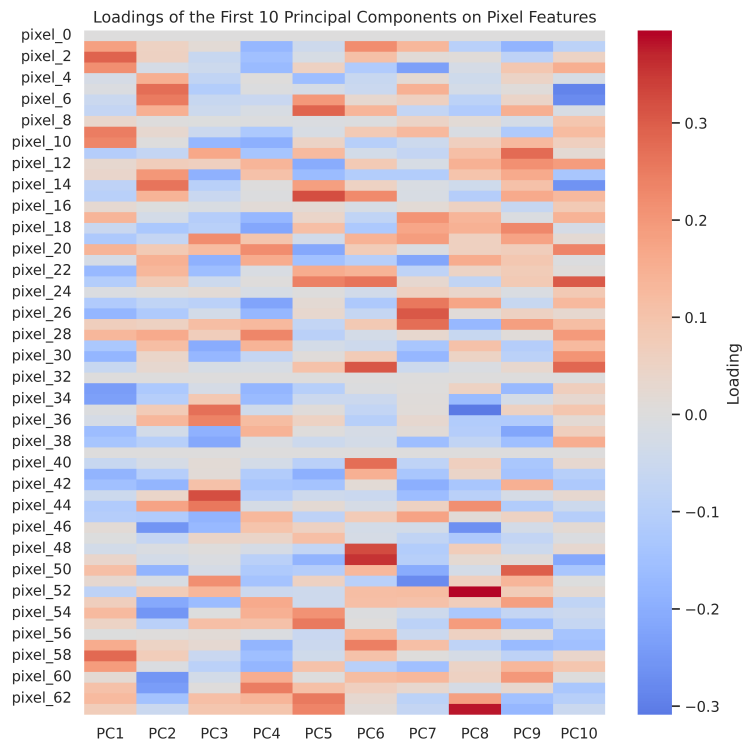


Figure 15: Loadings of the first ten principal components on the 64 pixel features

Inference: Each principal component draws on a broad, spatially coherent group of pixels rather than a single isolated one, confirming that the components are combining neighbouring, correlated pixels rather than simply reproducing the original feature axes. Later components have progressively more diffuse, lower-magnitude loadings, consistent with them capturing finer, less structured variation.

Table 3: PCA Variance Explained: Summary

Dataset	Setting	Components Retained	Explained Variance (%)	Justification
Digits	With-PCA	40 / 64	95.16	95% threshold keeps almost all discriminative pixel signal
Diabetes	With-PCA	8 / 10	99.15	Even a compact feature set has some redundant serum features

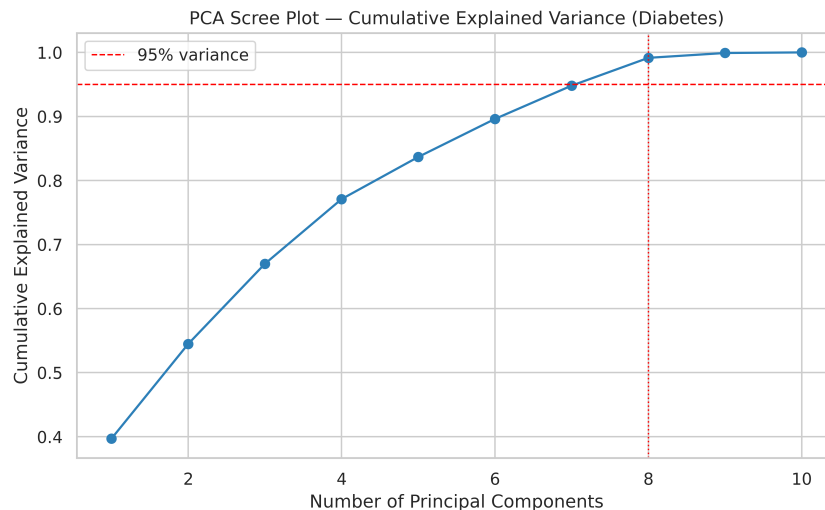


Figure 16: PCA scree plot: cumulative explained variance (Diabetes)

Inference: Because Diabetes has only 10 features to begin with, PCA can only offer a small reduction: 8 of 10 components are needed to reach 95% variance, confirming that the original feature set is already fairly efficient with limited internal redundancy. This sets the expectation that PCA’s impact on the regression results will be much smaller than its impact on the Digits classification results.

7 Machine Learning Algorithms

Ten classifiers were evaluated on the Digits dataset, exactly as specified in the assignment; each is summarised briefly below, with implementation notes rather than a full derivation of each algorithm’s mathematics. For the regression extension, the natural regressor counterpart of each classifier was used on the Diabetes dataset (for example **SVR** in place of **SVC**, or **Bayesian Ridge** as the closest available regression analogue of Naive Bayes).

7.1 Support Vector Machine (SVM)

Finds the maximum-margin hyperplane (or, with a kernel, a non-linear boundary) that separates classes with the largest possible gap.

- **Advantages:** effective in high-dimensional spaces, memory-efficient at inference (depends only on support vectors), flexible via the kernel trick.
- **Limitations:** training cost grows quickly with sample count, sensitive to the choice of C and γ , no direct probability output without extra calibration.

7.2 Naive Bayes

Applies Bayes’ theorem under a “naive” assumption that features are conditionally independent given the class; the Gaussian variant models each feature as normally distributed per class.

- **Advantages:** extremely fast to train and predict, works well with limited data, naturally probabilistic.
- **Limitations:** the independence assumption is rarely true for correlated pixel features, which visibly hurts its accuracy relative to every other model here.

7.3 k-Nearest Neighbours (KNN)

Classifies a new point by majority vote (optionally distance-weighted) among its k closest neighbours in the training set, under a chosen distance metric.

- **Advantages:** simple, no training phase, naturally handles multi-class problems, and adapts to locally complex decision boundaries.
- **Limitations:** prediction is slow on large datasets, sensitive to feature scale and to the curse of dimensionality, and k /metric/weighting all need tuning.

7.4 Logistic Regression

Models the log-odds of each class as a linear function of the (standardised) input features, fit by maximising the multinomial likelihood.

- **Advantages:** fast, interpretable coefficients, well-calibrated probabilities, a strong and cheap baseline.
- **Limitations:** assumes an (approximately) linear decision boundary in feature space, so it can underperform on problems with strongly non-linear class structure.

7.5 Decision Tree

Recursively splits the feature space on the feature/threshold pair that most reduces class impurity (Gini or entropy) at each node.

- **Advantages:** highly interpretable, handles non-linear boundaries, needs almost no pre-processing.
- **Limitations:** a single deep tree overfits easily and is unstable, since small changes in training data can produce a very different tree.

7.6 Random Forest

An ensemble of decision trees, each trained on a bootstrap sample with a random subset of features considered at every split; predictions are combined by majority vote.

- **Advantages:** much lower variance than a single tree, strong out-of-the-box accuracy, gives a feature-importance ranking for free.
- **Limitations:** less interpretable than a single tree, heavier to train and store, and can still overfit if trees are grown without any depth control.

7.7 AdaBoost

Trains a sequence of weak learners (shallow trees by default), reweighting misclassified samples after each round so later learners focus on the harder cases; the final prediction is a weighted vote.

- **Advantages:** often improves accuracy over a single weak learner, few hyperparameters to tune.
- **Limitations:** sensitive to noisy samples and outliers, since misclassified points keep receiving more weight; it was consistently the weakest ensemble on Digits in this experiment.

7.8 Gradient Boosting

Builds trees sequentially, where each new tree is fit to the residual (negative gradient of the loss) of the current ensemble, gradually reducing error.

- **Advantages:** typically strong accuracy, flexible loss functions, handles complex non-linear structure well.

- **Limitations:** slower to train than bagging-based ensembles, more hyperparameters (learning rate, depth, number of estimators) that interact with each other.

7.9 XGBoost

An optimised, regularised implementation of gradient boosting with second-order gradient information and built-in handling for speed and overfitting control.

- **Advantages:** fast even on modest hardware, strong regularisation reduces overfitting, consistently competitive across many problem types.
- **Limitations:** more hyperparameters to understand and tune than plain Gradient Boosting, and its gains over well-tuned Gradient Boosting were marginal on this dataset.

7.10 Stacking

Combines several diverse base learners (here: a Decision Tree, KNN and an SVM) by training a meta-learner (Logistic Regression) on their out-of-fold predictions.

- **Advantages:** can outperform any single base learner by combining their different strengths; was the best or joint-best model in both the No-PCA and With-PCA settings here.
- **Limitations:** more complex and slower to train and deploy, and its behaviour under PCA is only as good as its most PCA-sensitive base learner.

8 Experimental Setup

Python Version	3.12
Scikit-Learn Version	1.8
Other Libraries	pandas 3.0, NumPy 2.4, XGBoost 3.4, seaborn 0.13, matplotlib 3.10
IDE	Jupyter Notebook
Operating System	macOS Tahoe 26.6.1
Hardware	Apple Silicon (CPU + MPS GPU acceleration available)

9 Hyperparameter Selection

Every model was tuned with an exhaustive grid search (`GridSearchCV`) under 5-fold stratified cross-validation (Digits) or plain 5-fold cross-validation (Diabetes), optimising accuracy for classification and R^2 for regression. Each grid search was run twice per model – once on the standardised original features (No-PCA) and once on the PCA-reduced features (With-PCA), so that the best hyperparameters and not just the same fixed configuration are compared fairly in both settings.

Listing 2: Hyperparameter tuning and 5-fold cross-validation

```

1 def tune_and_cv(model, grid, X, y, cv_splits=5):
2     skf = StratifiedKFold(n_splits=cv_splits, shuffle=True, random_state=42)
3     gs = GridSearchCV(model, grid, cv=skf, scoring="accuracy",
4                       n_jobs=-1, refit=True)
5     gs.fit(X, y)
6     cv_scores = cross_validate(gs.best_estimator_, X, y, cv=skf,
7                               scoring=["accuracy", "f1_macro"], n_jobs=-1)
8     return gs.best_estimator_, gs.best_params_, cv_scores

```

Table 2: SVM Hyperparameter Tuning

Every combination of kernel, C and γ in the search grid was evaluated with 5-fold cross-validation. All twelve combinations are listed below, sorted by No-PCA accuracy.

Kernel, C , Gamma	Perf. (No-PCA)	Perf. (With-PCA)
rbf, $C = 1$, scale	0.9805	0.9805
linear, $C = 0.1$, scale	0.9798	0.9729
linear, $C = 0.1$, $\gamma = 0.01$	0.9798	0.9729
rbf, $C = 10$, scale	0.9791	0.9784
rbf, $C = 1$, $\gamma = 0.01$	0.9784	0.9763
rbf, $C = 10$, $\gamma = 0.01$	0.9784	0.9791
linear, $C = 1$, scale	0.9763	0.9701
linear, $C = 1$, $\gamma = 0.01$	0.9763	0.9701
linear, $C = 10$, scale	0.9763	0.9701
linear, $C = 10$, $\gamma = 0.01$	0.9763	0.9701
rbf, $C = 0.1$, scale	0.9255	0.9269
rbf, $C = 0.1$, $\gamma = 0.01$	0.9249	0.9255

Table 4: All 12 SVM grid combinations, mean 5-fold CV accuracy. Best: kernel = rbf, $C = 1$, gamma = scale.

Table 3: Naive Bayes Smoothing Choices

Smoothing Parameter (var_smoothing)	Performance (No-PCA)	Performance (With-PCA)
10^{-6}	0.8302	0.8901
10^{-7}	0.8038	0.8901
10^{-8}	0.7864	0.8901
10^{-9}	0.7662	0.8901

Table 5: All 4 smoothing values tested. Best: 10^{-6} for No-PCA, all four tie at 10^{-9} – 10^{-6} for With-PCA.

Table 4: KNN Hyperparameter Tuning

Full 16-combination grid (4 values of k , 2 weighting schemes, 2 distance metrics), sorted by No-PCA accuracy.

k	Weights	Distance Metric	Perf. (No-PCA)	Perf. (With-PCA)
5	distance	euclidean	0.9777	0.9750
3	uniform	euclidean	0.9763	0.9736
3	distance	euclidean	0.9763	0.9736
5	uniform	euclidean	0.9763	0.9722
3	distance	manhattan	0.9750	0.9701
3	uniform	manhattan	0.9743	0.9687
7	uniform	euclidean	0.9736	0.9708
7	distance	euclidean	0.9736	0.9736
9	distance	euclidean	0.9722	0.9701
9	uniform	euclidean	0.9701	0.9652
5	uniform	manhattan	0.9701	0.9701
5	distance	manhattan	0.9694	0.9715
7	distance	manhattan	0.9680	0.9680
7	uniform	manhattan	0.9659	0.9659
9	distance	manhattan	0.9659	0.9687
9	uniform	manhattan	0.9624	0.9645

Table 6: All 16 KNN grid combinations. Best: $k = 5$, weights = distance, metric = euclidean.

Best Hyperparameters: Remaining Classifiers

Model	Best Setting (No-PCA)	Best Setting (With-PCA)
Logistic Regression	$C = 1$, solver = lbfgs	$C = 1$, solver = lbfgs
Decision Tree	max_depth = 10, min_split = 2	max_depth = None, min_split = 2
Random Forest	200 trees, max_depth = 10	100 trees, max_depth = 20
AdaBoost	100 estimators, lr = 1.0	100 estimators, lr = 1.0
Gradient Boosting	150 estimators, depth 3, lr = 0.1	best of grid: $\{100, 150\} \times \{0.05, 0.1\} \times \{3, 5\}$
XGBoost	150 estimators, depth 5, lr = 0.1	best of grid: $\{100, 150\} \times \{0.05, 0.1\} \times \{3, 5\}$
Stacking	meta $C = 0.1$	meta $C = 1$

Table 7: Best hyperparameters found by grid search for the remaining classifiers.

10 Results

10.1 Classification (Digits)

Table 8: 5-Fold Cross-Validation Results: Summary (No-PCA vs With-PCA), Digits

Model	Avg Acc. (No-PCA)	Avg Acc. (With-PCA)	Avg F1 (No-PCA)	Avg F1 (With-PCA)
SVM	0.9805	0.9805	0.9805	0.9805
Naive Bayes	0.8302	0.8901	0.8302	0.8904
KNN	0.9777	0.9750	0.9776	0.9748
Logistic Regression	0.9694	0.9631	0.9693	0.9629
Decision Tree	0.8448	0.8191	0.8445	0.8176
Random Forest	0.9770	0.9631	0.9769	0.9628
AdaBoost	0.8017	0.7606	0.8016	0.7644
Gradient Boosting	0.9652	0.9360	0.9651	0.9359
XGBoost	0.9659	0.9464	0.9659	0.9463
Stacking	0.9882	0.9833	0.9882	0.9832

Table 9: Table 5: Full 5-Fold Cross-Validation Accuracy, No-PCA (Digits)

Model	Fold 1	Fold 2	Fold 3	Fold 4	Fold 5	Avg
SVM	0.9861	0.9757	0.9861	0.9721	0.9826	0.9805
Naive Bayes	0.8160	0.8507	0.8397	0.8084	0.8362	0.8302
KNN	0.9757	0.9896	0.9861	0.9617	0.9756	0.9777
Logistic Regression	0.9583	0.9722	0.9721	0.9652	0.9791	0.9694
Decision Tree	0.8472	0.8125	0.8571	0.8502	0.8571	0.8448
Random Forest	0.9722	0.9826	0.9791	0.9652	0.9861	0.9770
AdaBoost	0.8090	0.7986	0.7909	0.8397	0.7700	0.8017
Gradient Boosting	0.9514	0.9792	0.9686	0.9582	0.9686	0.9652
XGBoost	0.9583	0.9688	0.9582	0.9617	0.9826	0.9659
Stacking	0.9861	0.9861	0.9895	0.9895	0.9895	0.9882

Table 10: Table 5: Full 5-Fold Cross-Validation Accuracy, With-PCA (Digits, 40 components)

Model	Fold 1	Fold 2	Fold 3	Fold 4	Fold 5	Avg
SVM	0.9896	0.9757	0.9861	0.9686	0.9826	0.9805
Naive Bayes	0.8715	0.9028	0.9094	0.8502	0.9164	0.8901
KNN	0.9722	0.9722	0.9826	0.9721	0.9756	0.9750
Logistic Regression	0.9583	0.9688	0.9686	0.9477	0.9721	0.9631
Decision Tree	0.8056	0.8125	0.8641	0.8014	0.8118	0.8191
Random Forest	0.9549	0.9722	0.9756	0.9443	0.9686	0.9631
AdaBoost	0.7500	0.7639	0.7596	0.7387	0.7909	0.7606
Gradient Boosting	0.9062	0.9410	0.9547	0.9199	0.9582	0.9360
XGBoost	0.9271	0.9479	0.9617	0.9233	0.9721	0.9464
Stacking	0.9826	0.9896	0.9861	0.9721	0.9861	0.9833

10.2 Regression Extension (Diabetes)

Table 11: 5-Fold Cross-Validation Results: Summary (No-PCA vs With-PCA), Diabetes

Model	Avg R^2 (No-PCA)	Avg R^2 (With-PCA)	Avg RMSE (No-PCA)	Avg RMSE (With-PCA)
SVR (SVM)	0.4749	0.4742	55.68	55.71
Bayesian Ridge (NB analogue)	0.4795	0.4784	55.46	55.51
KNN Regressor	0.4008	0.4031	59.49	59.40
Linear Regression	0.4804	0.4772	55.39	55.56
Decision Tree Regressor	0.2846	0.1943	64.79	68.79
Random Forest Regressor	0.4199	0.4150	58.45	58.68
AdaBoost Regressor	0.4296	0.3964	58.17	59.84
Gradient Boosting Regressor	0.4264	0.3892	58.22	60.13
XGBoost Regressor	0.4277	0.3915	58.20	59.91
Stacking Regressor	0.4216	0.4182	58.32	58.68

Held-out test-set performance for the best model in each task, evaluated once on the untouched 20% test split:

Metric	Digits: Stacking (With-PCA)	Diabetes: Bayesian Ridge (With-PCA)
Accuracy / R^2	0.9806	0.4581
F1-macro / RMSE	0.9804	53.58
MAE (Diabetes only)	N/A	42.92

Table 12: Held-out test set results for the top model in each task.

11 Result Visualization

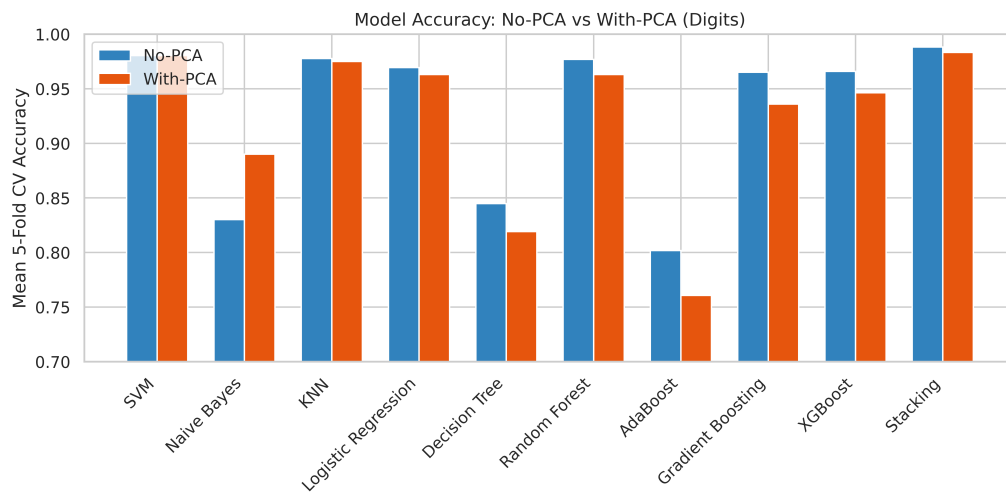


Figure 17: Mean 5-fold CV accuracy, No-PCA vs With-PCA, across all ten classifiers

Inference: Tree-based ensembles (Random Forest, Gradient Boosting, XGBoost, AdaBoost) lose a small but consistent amount of accuracy under PCA, while Naive Bayes is the only model that clearly improves, gaining nearly 6 points of accuracy once its features are decorrelated. SVM, KNN, Logistic Regression and Stacking are essentially unaffected either way, confirming that PCA’s impact here depends heavily on the type of model rather than being uniformly helpful or harmful.

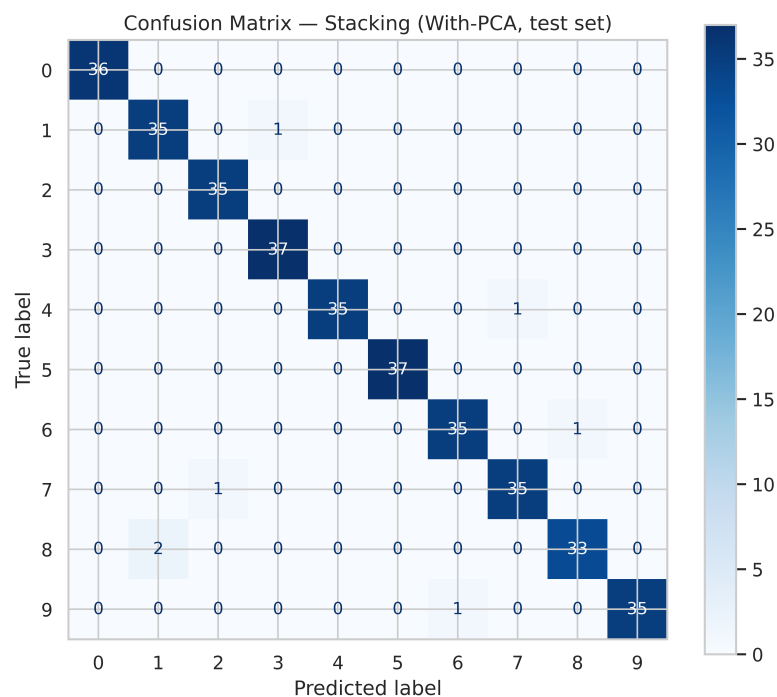


Figure 18: Confusion matrix: Stacking classifier (With-PCA), held-out test set

Inference: The confusion matrix is almost perfectly diagonal, with only a handful of misclassifications scattered across the 360 test samples (most notably two digit-8 samples predicted as 1, and single errors between a few other visually similar pairs). This is consistent with the

very high held-out test accuracy of 98.06% and confirms that the model generalises well beyond the cross-validation folds used for tuning.

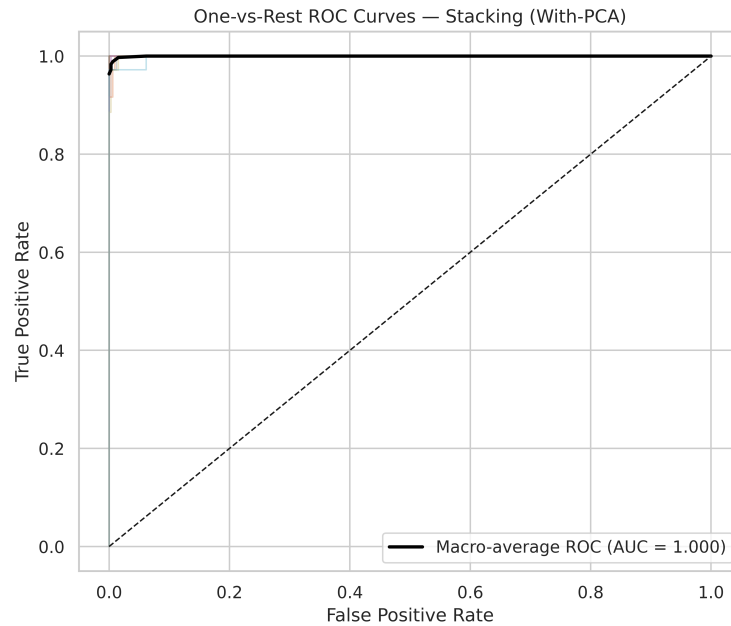


Figure 19: One-vs-rest ROC curves: Stacking classifier (With-PCA)

Inference: Every per-class ROC curve hugs the top-left corner, and the macro-averaged AUC is close to 1.0, indicating that the model separates each digit from the rest with very high confidence across almost the entire probability threshold range. This is expected given the already-strong accuracy and confusion-matrix results, and it confirms there is no single digit class that the model struggles to distinguish from the others.

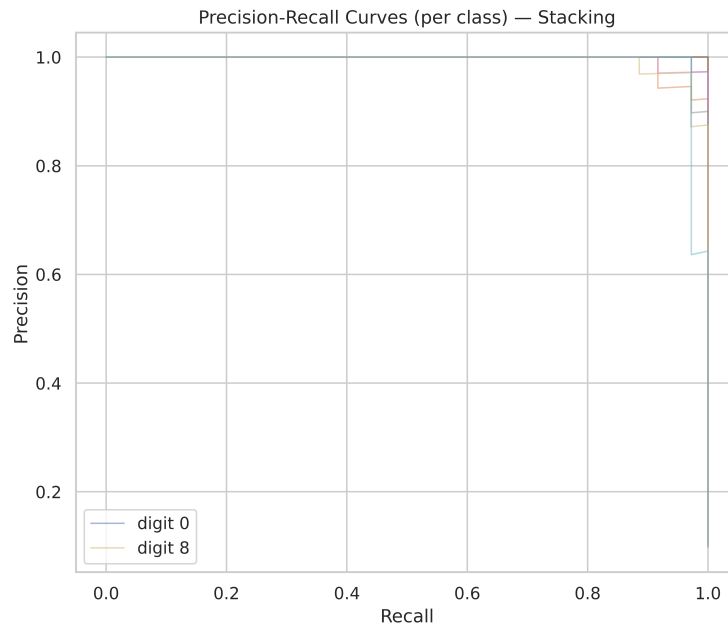


Figure 20: Precision-recall curves per class: Stacking classifier (With-PCA)

Inference: Precision stays close to 1.0 across almost the full recall range for every class, only dropping off sharply near recall = 1, which happens when the classifier is forced to also

capture its hardest, most ambiguous examples. This pattern is far more informative than the ROC curves for a task like this one, since it shows directly that very few false positives are introduced even when the model is pushed to recall nearly every true instance of a class.

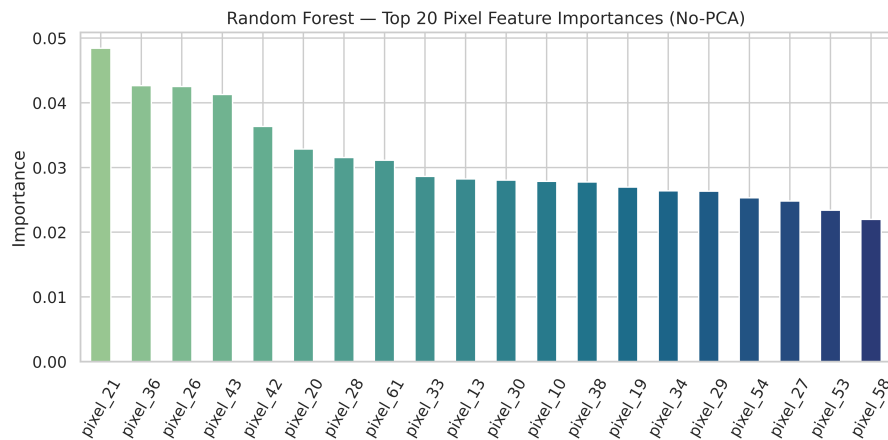


Figure 21: Random Forest: top 20 pixel feature importances (No-PCA)

Inference: Importance is concentrated in a subset of central pixels, matching the high-variance pixels identified during EDA, while border pixels contribute almost nothing. This overlap between “high variance” and “high importance” pixels reinforces why PCA, which is also driven by variance, captures most of the classification-relevant signal in a fraction of the original 64 dimensions.

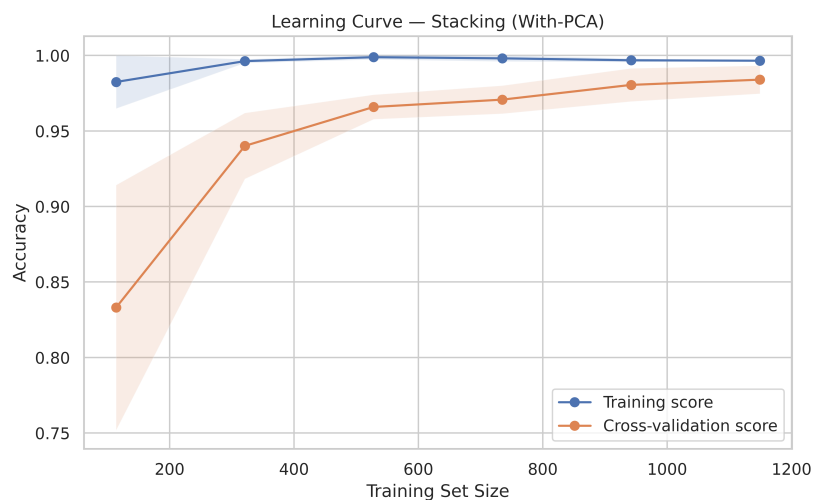


Figure 22: Learning curve: Stacking classifier (With-PCA)

Inference: The training and cross-validation curves converge closely as training-set size increases, with only a small persistent gap at full training size. This indicates the model is not strongly overfitting and that performance would likely improve only modestly, if at all, with additional training data. The current sample size is already sufficient for this model and feature space.

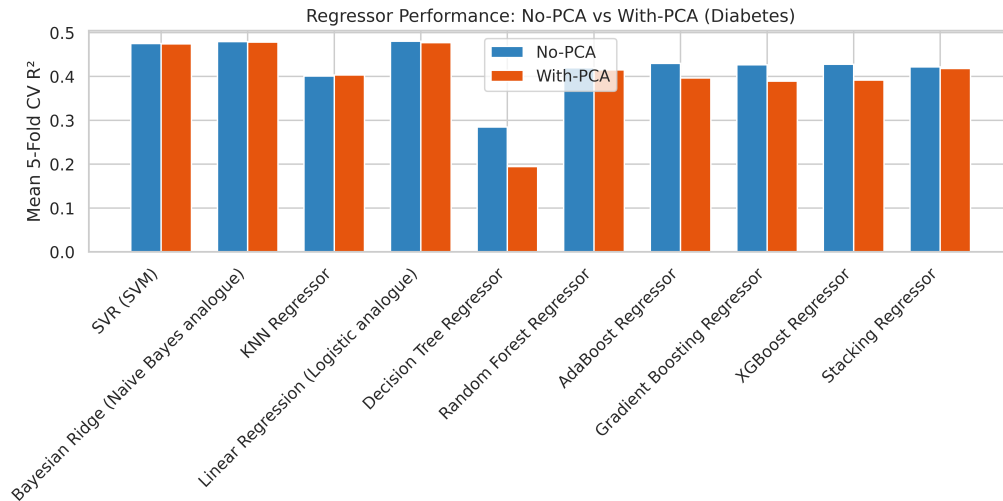


Figure 23: Mean 5-fold CV R^2 , No-PCA vs With-PCA, across all ten regressors (Diabetes)

Inference: Unlike the classification results, most regressors show only a small difference between the two settings, with the notable exception of the Decision Tree Regressor, which loses nearly 9 points of R^2 under PCA. Linear and kernel-based regressors (Linear Regression, SVR, Bayesian Ridge) are the most stable, since PCA components are themselves linear combinations of the mildly correlated original features and so barely change what a linear model can learn.

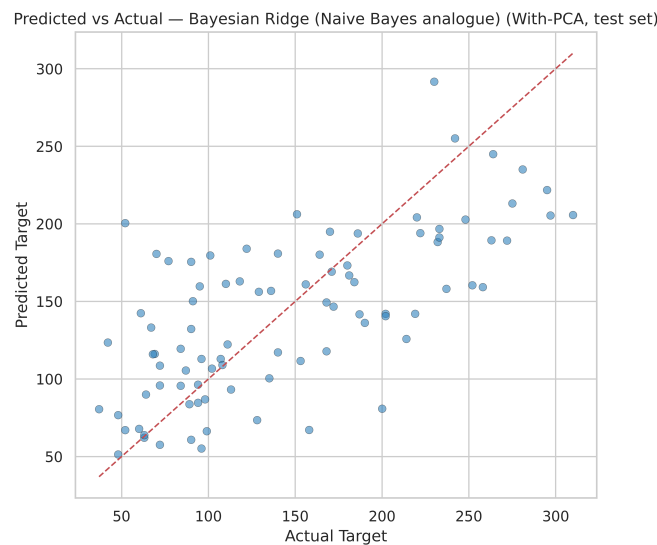


Figure 24: Predicted vs actual target: Bayesian Ridge (With-PCA), held-out test set

Inference: Predictions cluster reasonably around the diagonal but with substantial scatter, especially at higher actual target values where the model tends to under-predict. This is consistent with the moderate R^2 (0.46 on the test set) and reflects that only about half of the variance in disease progression is explained by these ten baseline measurements, and the remaining variance likely depends on factors not captured in this dataset.

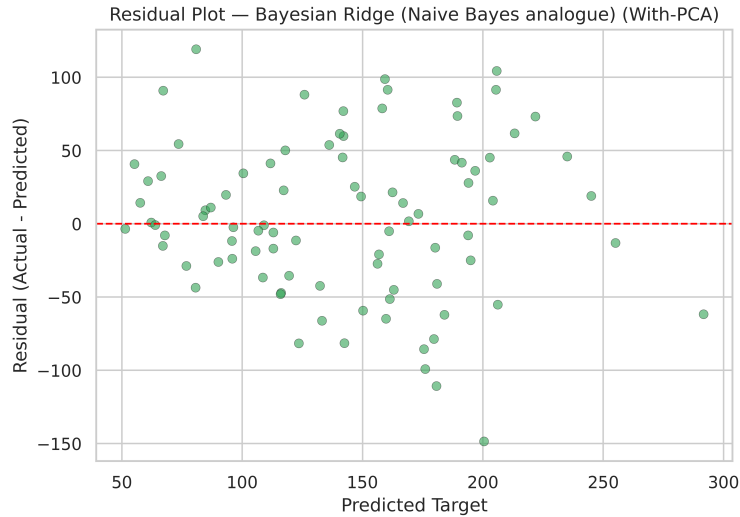


Figure 25: Residual plot: Bayesian Ridge (With-PCA)

Inference: Residuals are roughly centred around zero with no strong funnel or curved pattern, suggesting the linear-Gaussian assumptions behind Bayesian Ridge are a reasonable fit for this data. There is a slight tendency toward larger negative residuals (under-prediction) at higher predicted values, mirroring the pattern already visible in the predicted-vs-actual plot.

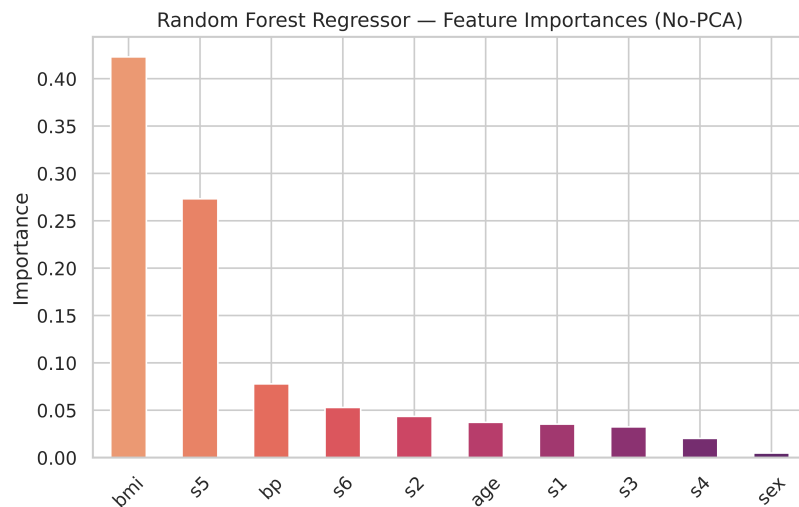


Figure 26: Random Forest Regressor: feature importances (No-PCA)

Inference: BMI and s5 dominate the importance ranking, matching the two features that showed the strongest correlation with the target during EDA, while age and s4 contribute the least. This agreement between correlation-based EDA and model-based importance gives confidence that the model is picking up genuine, interpretable physiological signal rather than spurious patterns.

12 Discussion

Across both experiments, PCA's benefit tracks closely with how large and internally correlated the original feature space is. On Digits (64 correlated pixel features), PCA gives Naive Bayes a clear boost by removing the feature correlations its independence assumption ignores, but it costs tree-based ensembles a small amount of accuracy since they already perform their own

implicit feature selection and lose some of the direct, per-pixel split information once features are rotated into components. SVM, KNN and Stacking are essentially unaffected either way. On the much smaller, already-compact Diabetes feature set, PCA's effect nearly disappears for every model except the Decision Tree Regressor, which is the most sensitive to any change in feature representation. Overall, PCA is best treated as a conditional tool, worth trying when the feature space is wide and correlated, and validated with cross-validation rather than assumed to help, rather than a default preprocessing step for every model and every dataset.

13 Conclusion

This experiment set out to measure the effect of PCA on ten classifiers, and extended that question to ten regressors on a second dataset. The results show a consistent story: PCA helps most for algorithms sensitive to feature correlation and dimensionality (notably Naive Bayes), offers little benefit either way for margin- or distance-based models (SVM, KNN, Stacking), and mildly hurts tree-based ensembles that already handle feature redundancy internally. The best overall classifier was Stacking (98.82% mean CV accuracy, 98.06% held-out test accuracy), and the best overall regressor was Bayesian Ridge ($R^2 = 0.4784$ mean CV, 0.4581 held-out test). PCA's usefulness scaled with the size and redundancy of the original feature space: substantial on the 64-feature Digits dataset, marginal on the already-compact 10-feature Diabetes dataset. This supports using PCA selectively and evaluating it empirically per model, rather than applying it uniformly across an entire modelling pipeline.

14 References

- [1] F. Pedregosa et al., “Scikit-learn: Machine Learning in Python,” *Journal of Machine Learning Research*, vol. 12, pp. 2825–2830, 2011.
- [2] T. Chen and C. Guestrin, “XGBoost: A Scalable Tree Boosting System,” in *Proc. 22nd ACM SIGKDD Int. Conf. on Knowledge Discovery and Data Mining*, 2016, pp. 785–794.
- [3] E. Alpaydin, C. Kaynak, “Optical Recognition of Handwritten Digits,” UCI Machine Learning Repository, 1998.
- [4] B. Efron, T. Hastie, I. Johnstone, R. Tibshirani, “Least Angle Regression,” *Annals of Statistics*, vol. 32, no. 2, pp. 407–499, 2004 (source of the Diabetes dataset).
- [5] I. T. Jolliffe, *Principal Component Analysis*, 2nd ed. New York: Springer, 2002.